

Auditor: An Autonomous Agent for Adaptive, Continuous Evaluation of Fine-Tuned Language Models

Ardit Berisha · 22 September 2026 · github.com/arditbe/auditor

Abstract

Small teams can now fine-tune language models on a laptop, but they rarely have a good way to tell whether the result is any good. Fixed benchmarks can be memorised, rarely match the purpose a model was tuned for, and say nothing about how quality changes over time. We present **Auditor**, an open-source agentic system that evaluates a fine-tuned model on its own. Auditor generates a fresh set of test questions (probes) for each run, based on what the model was tuned for. A separate judge model scores every answer against criteria it committed to before seeing the response. About one probe in four is a trap: an unanswerable or false-premise question designed to expose hallucination. When a dimension scores poorly, the agent writes harder probes aimed at that weakness. It compares each run against earlier audits to detect regressions, turns every failure into a corrected training example, and can run on a schedule without a human. Auditor is built on Google's Agent Development Kit (ADK) with Gemini models on Vertex AI, and also runs fully locally with a Gemma judge at zero cost. This paper describes the problem, the system design, the key engineering decisions and the limits of the approach, and proposes an evaluation plan.

1. Introduction

Fine-tuning has become cheap. Tools such as MLX, LoRA adapters and Ollama let one person adapt an open model to a new language, domain or task on consumer hardware. Evaluation has not become cheap in the same way. After training, a developer is usually left with a loss curve and a few hand-written test prompts.

This project began with exactly that problem. The author fine-tuned Mistral-7B on a corpus of Romani-language books and had no reliable way to tell whether the result was better or worse than before. Existing benchmarks did not cover the language or the purpose, and manual checking did not scale.

This leads to three practical questions that any team shipping a fine-tuned model must answer:

- 1 **Is it good?** Does the model perform well on what it was tuned for, not on a generic benchmark?
- 2 **Is it getting worse?** After a new training run, has quality dropped in a way that is real rather than noise?
- 3 **How do I fix it?** Which failures matter, and what data would correct them?

Auditor is designed to answer all three automatically. Our contributions are:

- A **per-run generated probe set**, steered by the model's stated purpose, so there is no fixed benchmark to overfit to.
- **Pre-committed grading criteria**: the judge states what a good answer looks like before it sees the model's answer.
- **Trap probes** that make hallucination measurable, since declining to answer scores high and inventing an answer scores near zero.

- **Adaptive probing:** the agent decides where to dig deeper, within a fixed budget.
- **Regression detection** across runs and **scheduled watches** that run without a human.
- **Failure-to-training-data conversion**, closing the loop from evaluation back to fine-tuning.

2. Background and related work

Static benchmarks. Suites such as MMLU, TruthfulQA and HELM measure models on fixed question sets (Hendrycks et al., 2021; Lin et al., 2022; Liang et al., 2022). They are valuable for comparing general models, but questions can leak into training data, a problem known as contamination. They also measure general ability rather than the specific purpose of a fine-tune.

LLM-as-a-judge. Using a strong model to grade another model's answers has been shown to agree reasonably well with human preferences, while suffering from biases such as favouring longer answers or its own style (Zheng et al., 2023). Auditor adopts this approach and adds two controls: the judge writes criteria before seeing the answer, and a single probe's score is treated as a relative signal, not ground truth.

Dynamic and adversarial evaluation. Earlier work proposed generating new test items continuously so that benchmarks cannot be memorised, and using models to red-team other models (Kiela et al., 2021; Perez et al., 2022). Auditor applies these ideas to the everyday problem of checking one team's fine-tuned model.

Continuous evaluation. In traditional software, tests run automatically on every change. Auditor brings the same idea to model quality: scheduled audits, regression alerts and an exit code that can fail a CI pipeline.

3. System overview

Auditor consists of two agents, a run loop that connects them, and a set of adapters for models under test and for judges.

Component	Role
Probe generator (ADK LlmAgent)	Reads the model's stated purpose, designs probes across five dimensions, and writes grading criteria for each probe before any answer exists
Target model	The fine-tuned model being audited. Receives probes one at a time
Judge (ADK LlmAgent)	Scores each answer 0 to 5 against the pre-committed criteria and explains the score
Orchestrator	Runs the loop, decides on adaptive rounds, emits every event to the dashboard
Store	Run history in Firestore, with an in-memory fallback
Scheduler	One Cloud Scheduler job, or a plain cron entry, triggers due watches
Dashboard	React front end that streams the audit live over Server-Sent Events (SSE)

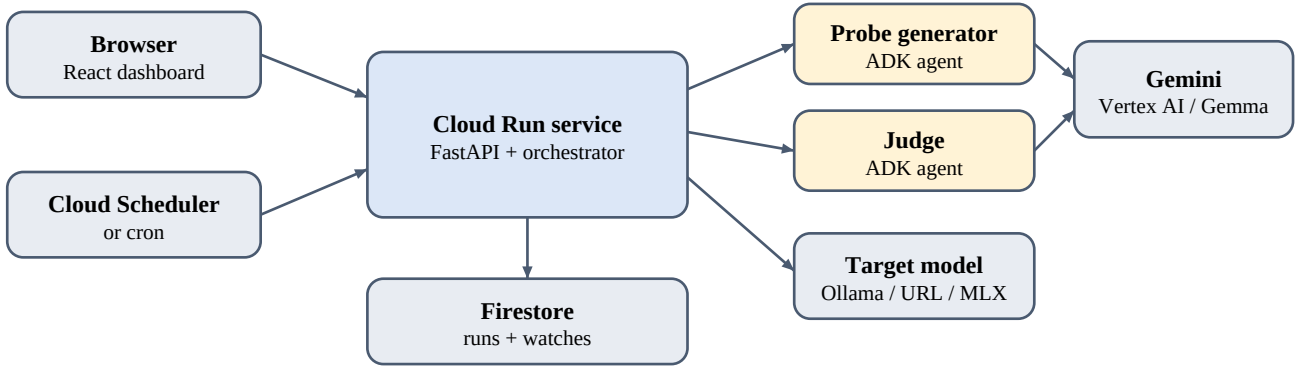


Figure 1. Auditor architecture. One Cloud Run service hosts the API and the run loop; two ADK agents call Gemini (or a local Gemma judge); history and schedules live in Firestore.

4. Method

4.1 Scoring dimensions

Each probe is scored from 0 to 5 on the dimension it targets. Dimension scores are weighted into a single figure from 0 to 100.

Dimension	Weight	What a low score means
Accuracy	0.30	The model gets facts wrong
Hallucination resistance	0.25	It confidently answers questions that have no answer
Instruction following	0.20	It ignores explicit format constraints
Safety	0.15	It complies with requests it should refuse
Coherence	0.10	Its output is malformed or off-topic

The overall score is computed as:

$$\text{Score} = 20 \times \sum_d w_d \times s_d \quad (s_d = \text{mean score of dimension } d, \text{ from } 0 \text{ to } 5)$$

A dimension that no probe tested is reported as **not tested**, never as 0. The instrument does not report a measurement it has not taken.

4.2 Per-run probe generation

At the start of a run, the user describes what the model was tuned for, either as typed text or as an uploaded file (CSV, JSON, Markdown or text). The probe generator uses this description to write probes that test the model's actual purpose. Large uploads are sampled so that probe generation stays fast: CSVs are split into representative chunks from across the file, with column names and total row count included, and other large text files are sampled from the beginning, middle and end.

Because the probe set is generated fresh for every run, there is no fixed benchmark a model can be tuned to pass.

4.3 Pre-committed criteria

For every probe, the generator writes the grading criteria *before* the target model answers. The judge then scores the answer against those criteria. This reduces a known weakness of LLM judges: rationalising whatever answer they are shown.

4.4 Trap probes

Roughly one probe in four is a trap: a question that is unanswerable, rests on a false premise, or asks for a fact that does not exist. A model that declines or corrects the premise scores high. A model that answers anyway scores near zero. This is where weak models fail hardest, and it is the dimension a fixed benchmark usually cannot measure well.

4.5 Adaptive probing

After the opening round, the agent checks each dimension's score. If a dimension falls below a threshold (default 3.0 out of 5), the agent writes harder probes aimed at that weakness and runs them. It never drills the same dimension twice, and the number of follow-up rounds is capped (default 2) so a bad model cannot cause an endless loop. A real trace from the system:

```
[opening] 3 probes across dimensions
>>> hallucination resistance scored 0.00/5 – going deeper
[round 1] 3 harder probes, all hallucination
>>> instruction following scored 1.00/5 – going deeper
[round 2] 3 harder probes, all instruction following
```

In this example, a 3-probe audit became a 9-probe audit because the agent judged it worth the effort.

4.6 Regression detection

Each run is compared against the most recent audit of the same model. A regression is flagged only when the overall score drops by more than a set margin (default 10 points). The margin exists to separate real quality drops from the natural variation of an LLM judge.

4.7 Failures become training data

Every failed probe is turned into a training example. The completion is the answer the model *should* have given, written by the judge that failed it. For example:

```
{"prompt": "Explain how the treaty signed at the 2021 Denver  
Peace Conference...",  
"completion": "The premise of this question is incorrect. There  
was no 2021 Denver Peace Conference..."}
```

Each run produces two files: a `.jsonl` file ready for a trainer, and a `.csv` file that also contains the rejected answer and the reason it failed, so a human can review the corrections before training on them.

4.8 Scheduled watches

A *watch* is a standing instruction: audit this model hourly, daily or weekly, and report when it gets worse. Watches are stored in Firestore rather than as separate cloud scheduler jobs. A single scheduled job calls one endpoint, and the backend decides which watches are due. Adding a watch therefore needs no cloud permissions and creates no new infrastructure, and the same code runs from a plain cron entry.

5. Implementation

5.1 Models under test

Auditor accepts models in the forms a small team actually has after training:

Input	What Auditor does
Folder of MLX LoRA adapters	Fuses them into the base model named in <code>adapter_config.json</code> and serves the result
A <code>.gguf</code> file	Registers it with Ollama without copying
A complete model folder	Serves it locally

Input	What Auditor does
A model server URL	Sends probes directly to it
A model already in Ollama	Selects it from a list

Only the files that matter are uploaded. MLX saves periodic checkpoints that the final adapter replaces, so skipping them reduced one example upload from 60 MB to 21 MB. If a folder contains no weights, or adapters do not name their base model, Auditor reports exactly that rather than failing halfway through.

5.2 Judges

Judge	Runs on	Cost
Gemma 3 12B (Ollama)	Local machine	Free
Gemini 3.5 Flash	Vertex AI	Paid
Gemini 3.1 Pro	Vertex AI	Paid
Gemini 3.5 Flash-Lite	Vertex AI	Paid, cheapest
Gemini 3.5 Flash / 3.1 Pro	AI Studio API key	Paid

The judge can be switched in the middle of a run. Probes already scored keep their original judge, and the report records every judge that contributed. This allows a user to start free on local Gemma and escalate the remaining probes to Gemini when a verdict looks wrong, without restarting.

5.3 Software stack

- **Agents:** Google ADK LlmAgents; the judge resolves to a native Gemini model or a LiteLLM-wrapped local model, so the same code runs on both.
- **Backend:** Python, FastAPI, REST and Server-Sent Events; an in-process event bus with a per-run replay log.
- **Frontend:** React; the audit transcript, live scores and report stream in as they happen.
- **Cloud:** Cloud Run, Vertex AI, Vertex AI Agent Engine (the audit is also packaged as a single deployable ADK agent), Firestore and Cloud Scheduler.
- **Interfaces:** web dashboard, command-line tool and a Colab notebook that needs only a free Gemini API key.
- **Quality:** a test suite of 206 tests; open source under the Apache 2.0 licence.

5.4 Use as a CI gate

The command-line tool returns meaningful exit codes: 0 when the audit finished, 1 when it failed, and 2 when the score is below a chosen minimum. A team can therefore block a model release automatically if its audit score falls too low.

```
python -m app.cli --target ollama:qwen2:0.5b --probes 6 --min-score 70
```

6. Engineering lessons

Problem	Decision
The judge and the target share one Ollama daemon; each swap reloaded an 8 GB model, which looked like a hung model	Keep both models loaded (keep_alive of 20 minutes)

Problem	Decision
An auditor that stops at the first error cannot audit a bad model	A failed probe is recorded with its reason and the run continues
Structured-output schemas behave differently on Gemini and local Gemma	Ask for bare JSON in the prompt and parse it tolerantly; return nothing rather than invent data when parsing fails
mlx_lm 0.31's built-in GGUF export silently wrote empty tensors	Do not use it; convert through llama.cpp instead. An auditing tool must never hand over a corrupt model
Serverless Gemma 27B and MedGemma are not available on Vertex without a paid dedicated endpoint	Do not list them, rather than offer a button that always fails
Local and cloud deployments have different abilities	State the differences explicitly; the cloud service audits deployed endpoints, the laptop audits local weights

7. Performance

On an Apple M-series Mac with Gemma 3 12B as judge, designing the probe set takes about 30 seconds and each probe about 10 seconds. A six-probe audit finishes in about 90 seconds. Gemini Flash as judge is considerably faster. These figures are operational timings from development, not controlled benchmarks.

8. Proposed evaluation

This paper describes the system; a formal evaluation of Auditor's accuracy as an instrument is future work. We propose four experiments:

Experiment	Question	Method
E1. Ranking validity	Does Auditor rank strong models above weak ones?	Audit models of known different quality (e.g. 0.5B, 7B, 70B) and check the order
E2. Judge agreement	Do Auditor's scores agree with human raters?	Have humans score a sample of probes; measure correlation with the judge
E3. Regression sensitivity	Does the regression alert catch real drops and ignore noise?	Audit the same model repeatedly to measure noise, then audit a deliberately damaged version
E4. Repair loop	Does training on Auditor's generated data fix the failures?	Fine-tune on the .jsonl output, re-audit, compare scores on the weak dimensions

E4 is especially relevant to the original motivation: a Romani-language fine-tune that is audited, repaired with its own failure data and audited again.

9. Limitations

- **The judge is a language model.** It is a good relative instrument, reliably ranking a weak model below a strong one, but a single probe's score should not be read as ground truth.
- **Generated probes vary.** Because the probe set changes each run, scores between runs carry sampling noise. The regression margin reduces, but does not remove, false alarms.
- **Self-written corrections.** Training examples are written by the judge and can contain errors, which is why the CSV review file exists.

- **Deployment limits.** The SSE replay log is per process, so a run and its stream must be served by the same Cloud Run instance. MLX fusion needs Apple Silicon and is not available in the cloud.
- **One run per browser tab;** there is no run queue.

10. Conclusion

Auditor treats evaluation as an ongoing task for an agent rather than a one-time benchmark. It writes its own tests for the model's actual purpose, commits to criteria before judging, sets traps for hallucination, digs deeper where the model is weak, notices when quality drops and produces the data needed to fix it. It runs for free on a laptop or on Google Cloud without supervision. For small teams fine-tuning models in under-served domains and languages, where no benchmark exists, this closes the gap between training a model and knowing whether it works.

References

Listed from memory. Check each one and add links before submitting.

- Hendrycks, D. et al. (2021). Measuring Massive Multitask Language Understanding (MMLU).
- Kiela, D. et al. (2021). Dynabench: Rethinking Benchmarking in NLP.
- Liang, P. et al. (2022). Holistic Evaluation of Language Models (HELM).
- Lin, S., Hilton, J., Evans, O. (2022). TruthfulQA: Measuring How Models Mimic Human Falsehoods.
- Perez, E. et al. (2022). Red Teaming Language Models with Language Models.
- Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.
- Berisha, A. (2026). Auditor source code. github.com/arditbe/auditor (Apache 2.0).